

Fraud Detection in Food Delivery Apps

*Comparison of Machine Learning models for identifying
fraudulent orders on food delivery platforms*

MODELS EVALUATED

8

ALGORITHMS

DT, RF, XGB, LR, GB, SVM

METRICS

Precision Recall F1 AUC

Context & Problem

The Problem

- > Online fraud through social engineering attacks
- > Fraudsters use stolen credentials to access legitimate accounts
- > Purchases made with stolen cards — no real personal data used
- > Fraudsters buy resellable items (e.g. beverages) for profit
- > Victim disputes the charge -> Chargeback (CBK) hits the merchant

Financial Impact (Brazil 2021)

- > BRL 5.8 Billion in blocked fraud (ClearSale)
- > Chargeback = largest financial loss in e-commerce
- > Healthy market approval rate: 95-97%
- > Acceptable CBK rate in the market: 0.7%

Objective

- > Train an ML model to flag potentially fraudulent orders
- > Minimize false positives (blocking legitimate orders)
- > Maximize fraud capture rate (Recall)
- > Beat the market benchmark (95% approval, 0.7% CBK)

Data Collected

Food delivery orders dataset | Feb-Mar 2022

Order Variables

- > pedido_id -- Unique order ID
- > pedido_valor_total -- Amount paid
- > pedido_valor_bruto -- Gross value (no discount)
- > pedido_item_qtd -- Number of items
- > pedido_registro -- Date and time
- > Dia_Da_Semana -- Day of the week

Customer Variables

- > cliente_dias_cadastro -- Days since signup
- > cliente_dias_ultimo_pedido -- Days since last order
- > cliente_pedidos_dinheiro_qtd -- Cash orders count
- > cliente_pedidos_cancelados_qtd -- Cancelled orders
- > cliente_pedidos_valormedio_seismeses -- 6-mo avg spend
- > cliente_dispositivo_plataforma -- Device platform

Payment Variables

- > pedidomeio_bin -- First 6 card digits (BIN)
- > pedidomeio_nro -- Last 6 card digits
- > subtipo -- Card brand / subtype
- > pedidomeio_nometit -- Name on card
- > primeiro_nome_igual -- Name matches account?
- > bin_mp -- Positive purchase history on BIN

Address / Access Variables

- > pedido_endereco_cidade / uf / cep
- > pedido_endereco_recorrencia -- Address reuse count
- > UF_CPF = UF_ENTREGA -- State consistency check
- > pedido_ip -- IP address at purchase
- > device_mp -- Positive history on device
- > email_mp / telefone_mp -- Positive history

Target Variable: qt_cbk = 1 (Chargeback / Fraud occurred) | 0 (Legitimate order) -- ~1% fraud rate in the original dataset (363,257 orders)

Methodology

1

COLLECT

Real delivery app
data
Feb-Mar 2022
363k orders

2

CLEAN

Monetary value
formatting
Recurrence
variables per day

3

EDA

Exploratory
data analysis
Feature distributions
~1% fraud rate

4

FEATURES

One-hot encoding
Engineered variables
cancellation_rate
high_risk_hour

5

BALANCE

OverSampling
UnderSampling
class_weight
scale_pos_weight

6

MODELS

8 algorithms
tested and
compared
with full metrics

Evaluation Metrics: Precision (primary) · Recall · F1-Score · AUC-ROC · Accuracy · Confusion Matrix
Split: 80% train / 20% test (stratified) + Cross-Validation StratifiedKFold (k=5)

Feature Engineering

Creating and transforming variables to enrich the model

Engineered Variables (created)

- > cancellation_rate = cancellations / previous orders
- > hour -- Order hour extracted as integer (0-23)
- > high_risk_hour -- 1 if order placed between 10pm-5am
- > new_customer -- 1 if no order history exists
- > rec_email -- Orders from same email on the same day
- > rec_card -- Orders from same card on the same day
- > rec_device -- Orders from same device on the same day
- > same_address_enc -- Binary: address used before

One-Hot Encoding

- > Day_Of_Week -> 7 columns
- > email_domain -> top 20 + OTHERS
- > card_subtype (brand) -> multiple
- > customer_device_platform
- > order_delivery_type
- > delivery_state (UF) -> 27 states

Top Features Identified (Random Forest)

- > #1 rec_email -- Email recurrence on the same day
- > #2 bin_mp -- Positive purchase history on card BIN
- > #3 rec_device -- Device recurrence on the same day
- > #4 DESKTOP platform -- Atypical purchase channel
- > #5 customer avg spend (last 6 months)
- > #6 rec_card -- Card recurrence on the same day

Total features in final model: 183 columns (after one-hot encoding) · Complete dataset, no missing values

Machine Learning Models Used

01 Decision Tree

Hierarchical splits on features. `max_depth=5`, `criterion=entropy`. Starting point -- high interpretability.

03 Balanced Random Forest

Same Random Forest with `class_weight='balanced'`. Penalizes errors on the minority class (fraud) more heavily.

05 Logistic Regression

Linear probabilistic model with regularization. Identifies most significant variables ($p < 5\%$).

07 Gradient Boosting

`sklearn GradientBoostingClassifier`. Trains sequential trees correcting residual error at each iteration.

02 Random Forest

Ensemble of decision trees with bootstrap. `n_estimators=100`. Reduces overfitting from a single tree.

04 Random Forest + HyperParam Tuning

`RandomizedSearchCV` (30 iterations, `CV=3`). Automatic search over `n_estimators`, `max_depth`, `max_features`, `min_samples`, `bootstrap`.

06 XGBoost

Gradient Boosting with trees. `scale_pos_weight=neg/pos`. `max_depth=9`. Achieved highest Precision in study (98%).

08 SVM (RBF)

Support Vector Machine with radial basis function kernel. `class_weight='balanced'`. Requires feature scaling.

Comparative Results Across Models

Original dataset: 363,257 orders | 637 frauds in test set (72,652 total) | Source: research data

Model	Source	Precision	Recall	F1	TP	FP	TN	Accuracy
RF + HyperParam Tuning	TCC p.31	95%	36%	53%	232	11	72,004	99.4%
XGBoost	TCC p.35	98%	7%	13%	45	1	72,014	99.2%
RF + OverSampling	TCC p.28	3%	73%	6%	463	14,558	57,457	79.7%
Decision Tree	TCC p.20	63%	3%	5%	17	10	72,005	99.1%
Logistic Regression	TCC p.34	49%	3%	6%	21	22	71,993	99.1%

Accuracy Paradox (imbalanced data): with only 0.9% fraud rate, even a model that blocks ZERO frauds achieves 99.1% accuracy — so Accuracy is misleading here.
RF + OverSampling is the only model where Accuracy drops to 79.7% because it generated 14,558 false positives, blocking 20% of legitimate orders.
=> Precision (fraud class) is the primary metric: how many flagged orders are truly fraudulent.

All confusion matrices from TCC research data TP = Frauds correctly blocked FP = Legitimate orders incorrectly blocked Sorted by Precision

Class Balancing & Hyperparameter Tuning

Strategies to handle the imbalanced dataset (~0.9% fraud rate)

OverSampling (RandomOverSampler)

- > Replicates samples from the minority class
- > Balanced training: 288,010 vs 288,010
- > Recall increased from 3% to 80%
- > Precision dropped from 63% to 2%
- > Too many legitimate orders blocked
- > -> NOT a good choice for this problem

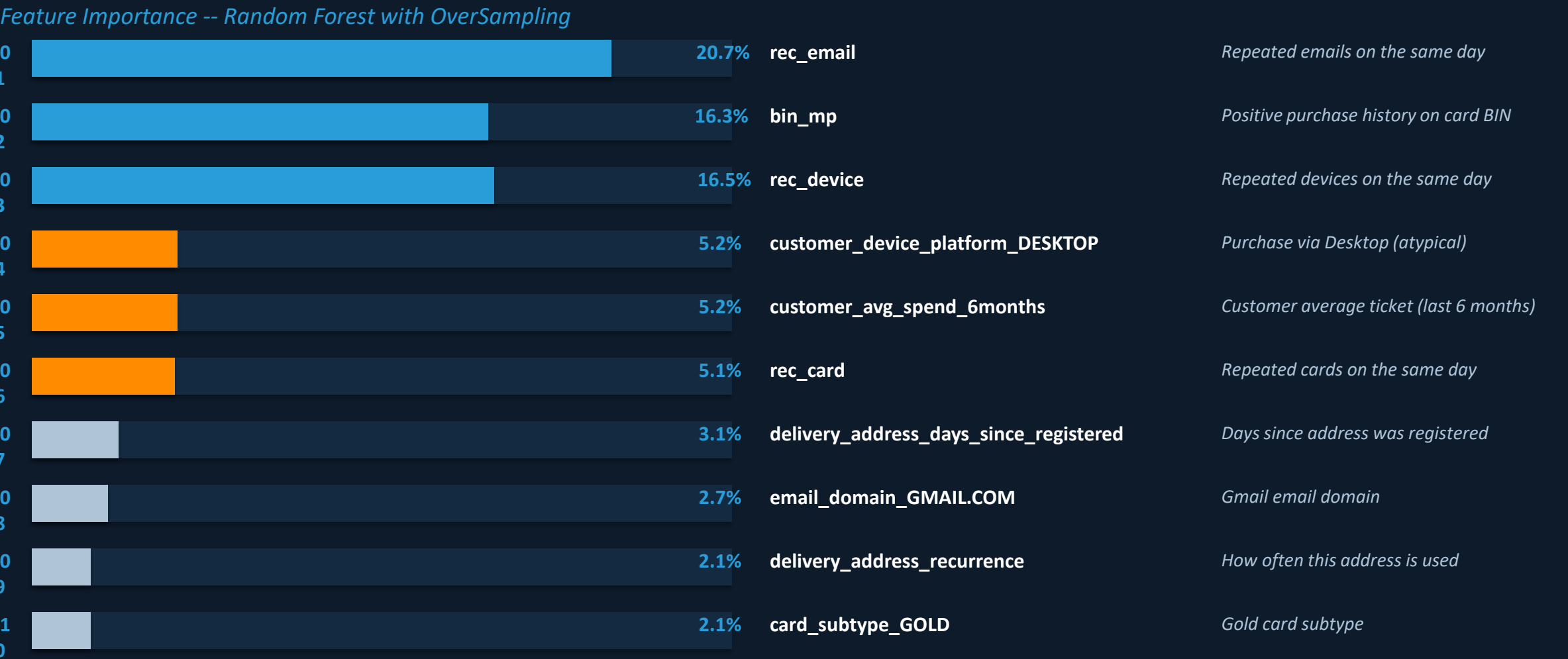
UnderSampling (RandomUnderSampler)

- > Reduces majority class to minority class size
- > Training set: 2,595 vs 2,595 orders
- > Recall: 79% | Precision: 2%
- > Worse results than OverSampling
- > Loses too much legitimate class information
- > -> NOT a good choice for this problem

RandomizedSearchCV (Hyperparameters)

- > 30 random parameter combinations
- > CV=3 internal folds, scoring='f1'
- > Search grid explored:
- > n_estimators: 50-2000
- > max_depth: 5 to 50 + None
- > max_features: auto, sqrt
- > min_samples_split: 2, 5, 10
- > bootstrap: True, False
- > Best parameters found:
- > n_estimators=50, bootstrap=False
- > max_features='sqrt', max_depth=None
- > -> Precision 95%, Recall 36%, F1 53%

Most Important Features for Fraud Detection



Conclusions & Final Results

BEST MODEL: Random Forest with Hyperparameter Tuning

PRECISION

95%

RECALL

36%

F1-SCORE

53%

AUC-ROC

0.96

Benchmark Comparison

- > Market standard: 95% approval, 0.7% CBK rate
- > Model reached 95% Precision -- matches benchmark
- > Legitimate order approval: 99.6% (above market)
- > Recall of 36%: catches 252 out of 637 frauds
- > Model is one layer in a multi-layered defense
- > Call center validates higher-risk flagged orders

Algorithm Insights

- > XGBoost: highest Precision (98%), lowest Recall (7%)
- > Balanced RF: highest Recall (80%), lowest Precision (2%)
- > Tuned RF: best overall balance (F1=53%)
- > OverSampling hurt Precision significantly
- > UnderSampling performed even worse than OverSampling
- > Logistic Regression < RF and XGBoost on this problem

Next Steps

- > Retrain monthly to adapt to new fraud patterns
- > Add velocity checks (orders per hour per user)
- > Implement a second authentication factor
- > Integrate model score with manual review desk
- > Test LightGBM and CatBoost
- > Explore anomaly detection (Isolation Forest)